

Context and memory

Module 3 · how knowledge reaches the agent

Lab: L01 First session and memory

OBJECTIVES

What you will be able to do

- Decide what belongs in an instruction file, a rule, auto memory, or neither
- Write a CLAUDE.md or AGENTS.md that changes behaviour
- Use AGENTS.md across several folders, and choose which files load
- Scope rules to part of the codebase
- Check what actually loaded
- Keep the context budget healthy

Two ways Claude remembers

Instruction files

Who writes it: You

Holds: Rules, commands, conventions

Scope: Project, user, or organisation

Loaded: Every session

vs

Auto memory

Who writes it: Claude

Holds: Learnings from your corrections

Scope: One repository, on this machine

Loaded: Every session (first 200 lines or 25 KB of the index)

Both are context, not enforcement. Claude usually follows them. If something must always happen, use a hook (M9) or a permission rule (M8).

Instruction files: load order, broadest first

1 Organisation (managed)

everyone in the organisation

`Windows C:\Program Files\ClaudeCode\CLAUDE.md · macOS /Library/Application Support/ClaudeCode/CLAUDE.md · Linux /etc/claude-code/CLAUDE.md`

2 User

only you, all projects

`~/.claude/CLAUDE.md`

3 Project

the team, via git

`./CLAUDE.md or ./.claude/CLAUDE.md`

4 Local

only you, this project

`./CLAUDE.local.md (add to .gitignore)`

The most specific file is read last. An organisation can also ship the managed text inside managed settings (the claudeMd key).

What loads when

At launch

Files in the folder where you start Claude and every folder above it. Concatenated, not overridden

On demand

A CLAUDE.md in a subfolder loads when Claude reads or edits a file there

Confirm what loaded

Run /context and look under Memory files, or open /memory

TWO DETAILS THAT SURPRISE PEOPLE

Large on-demand files are cut

A subfolder CLAUDE.md or path rule over 40,000 characters (200K context window) arrives cut off, with a note to read the rest. Check with wc -m.

/add-dir folders don't load theirs

A folder added with /add-dir or --add-dir gives file access only. To load its CLAUDE.md too, start Claude with:

```
CLAUDE_CODE_ADDITIONAL_DIRECTORIES_CLAUDE_MD=1
```

AGENTS.md: what Claude reads

Many teams keep one AGENTS.md that several coding tools read. Claude Code reads it too (v2.1.277 or later).

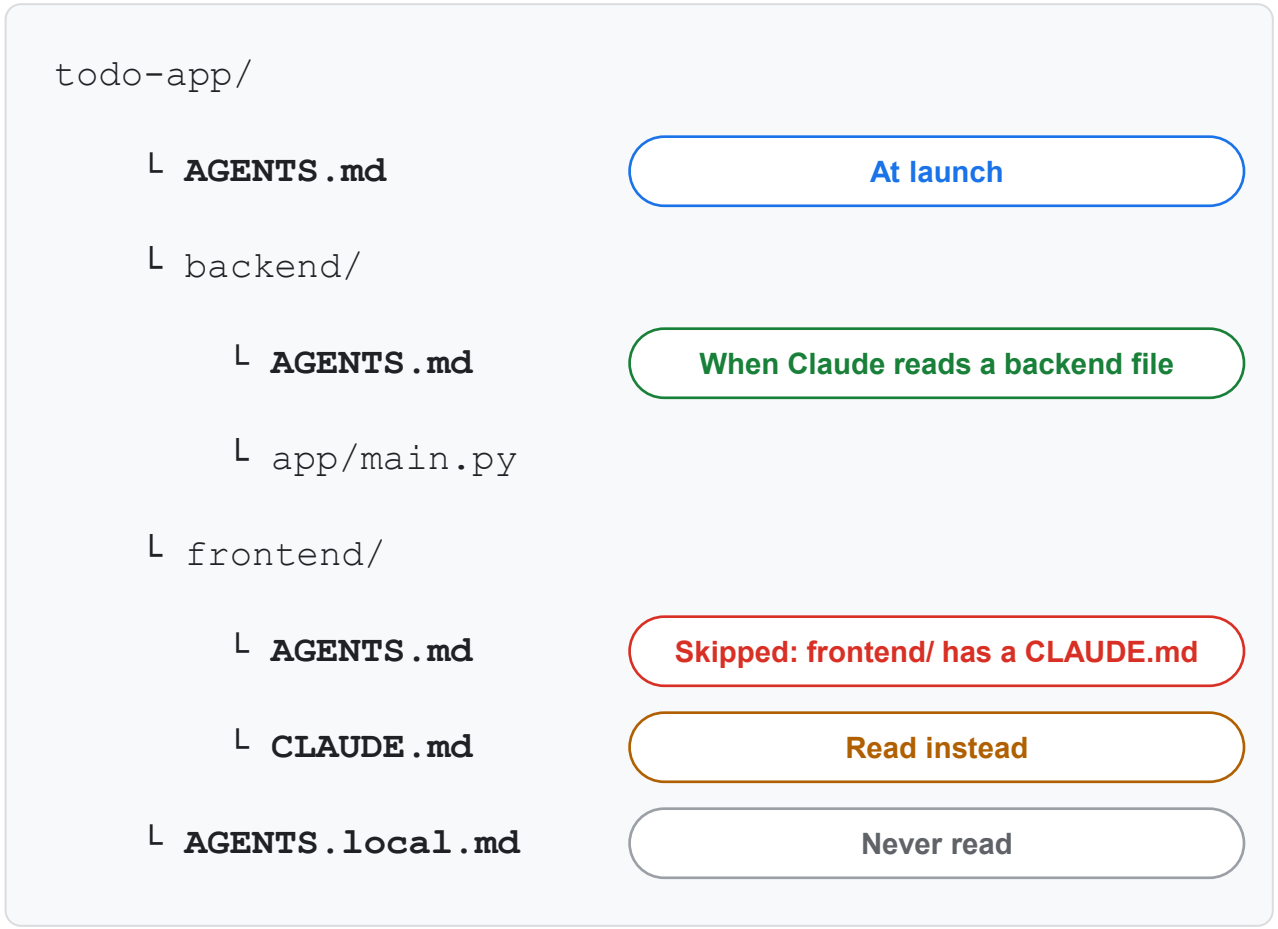
YOUR REPO HAS	CLAUDE READS
AGENTS.md only	AGENTS.md
AGENTS.md and a CLAUDE.md (in the folder or above)	CLAUDE.md only
AGENTS.md and a CLAUDE.local.md	CLAUDE.local.md only (a common trap)
A CLAUDE.md that contains @AGENTS.md	Both, through the import

```
@AGENTS.md

## Claude Code
Use plan mode for changes under
backend/app/rules.py.
```

A CLAUDE.md next to it: imports the shared file, then adds Claude-only lines. Works on every version.

AGENTS.md in more than one folder



At session start

Every AGENTS.md and .claude/AGENTS.md in the start folder and the folders above it

In subfolders

Loads when Claude reads a file in that folder, unless the folder has its own CLAUDE.md

Never read

AGENTS.local.md, AGENTS.override.md, anything under .agents/

Choose which files load: Project instructions

Open /config and set Project instructions. It works for AGENTS.md in every folder.

VALUE	CLAUDE READS
`claude-md-or-agents-md`	CLAUDE.md files, or AGENTS.md when there is no CLAUDE.md or CLAUDE.local.md. The default
`claude-md-and-agents-md`	Both. In each folder, CLAUDE.md first, then AGENTS.md. Never reads the same file twice
`claude-md`	CLAUDE.md files only
`managed-only`	Only the organisation's managed file and auto memory at launch



Keeping a CLAUDE.local.md in an AGENTS.md project? Set claude-md-and-agents-md so you get both.

AGENTS.md: version, Windows and this course

Version

Reading AGENTS.md directly needs v2.1.277 or later; some sessions (Bedrock, telemetry off) need v2.1.281

Windows

Prefer the @AGENTS.md import to a symlink: symlinks need Developer Mode and Git may check them out as plain text

In this course

One AGENTS.md per folder that needs it, plus a CLAUDE.md import only if /context does not list it

QUICK CHECK AFTER CREATING THE FILE

```
claude --version          # 2.1.277 or later?
/context                 # AGENTS.md under Memory files?
```

Add to the file when...

Same mistake twice

Claude makes the same mistake a second time

Review catch

A code review catches something Claude should have known

Same correction

You type the same correction you typed last session

New teammate

A new teammate would need the same context

Don't write the file in advance. Grow it, one line per real problem.

What belongs in it, and what does not

Put here

- Build, run, test commands
- Project layout in a few lines
- Conventions that differ from the tool defaults
- Business rules Claude cannot guess
- "Always do X" / "never do Y", with the reason

Do not put here

- Long multi-step procedures (skill)
- Rules for one folder only (path rule)
- Facts Claude can read from the code itself
- Anything that must be enforced (hook)
- Secrets

M00: explain the why. A rule with its reason lets Claude handle cases the rule never mentioned.

Write rules a reviewer could check

M00: quantify what you mean. The same applies to every line in an instruction file.

VAGUE	VERIFIABLE
"Format code properly"	"Use 4-space indentation in Python, 2-space in React (JSX)"
"Test your changes"	"Run uv run pytest -q in backend/ before saying a change is done"
"Keep files organised"	"API routes live in backend/app/main.py; business rules only in backend/app/rules.py"
"Be careful with dates"	"Store and return dates in UTC, ISO 8601. due_date is YYYY-MM-DD"

Keep it short

Under 200 lines

Target under 200 lines per file. Longer files cost context and lower adherence

Headers and bullets

Use headers and bullets, not paragraphs

No conflicts

If two rules contradict, Claude may pick either. Remove conflicts

Notes for humans

Leave notes in <!-- HTML comments -->; they are stripped before the file reaches Claude

Rules that load only when needed

Put topic files in .claude/rules/. Add paths: to load a rule only when Claude works with matching files.

.CLAUDE/RULES/BACKEND.MD

```
---
paths:
  - "backend/**/*.py"
---

# Backend rules
- Dates are UTC, ISO 8601.
- New endpoints return the same error
  shape as the existing ones.
```

FRONTMATTER	EFFECT
none	Loads at launch, like the main file
`paths:`	Loads when Claude reads, writes or edits a matching file



Working on the frontend? These backend rules use no context at all.

Path rules: patterns and scope

Globs

Relative to the folder that contains .claude: **/*.py, frontend/src/**/*, frontend/src/**/*.{js,jsx}

One field

paths is the only frontmatter field Claude Code reads; others are ignored

Personal rules

Use ~/.claude/rules/ for personal rules across all projects

Monorepos

Skip other teams' CLAUDE.md files with the claudeMdExcludes setting

Auto memory

Claude takes notes for itself as you work. Each note is a Markdown file with a type.

user

Your role, expertise, working preferences

feedback

Corrections you gave and approaches you confirmed

project

Ongoing work, deadlines and decisions not in the code

reference

Where to find things outside the project

It skips **anything it can read from the code** (architecture, file paths) and anything your CLAUDE.md already says.

Auto memory is for your habits. Team rules belong in the committed file.

Auto memory: where it lives and how to control it

Stored in

~/.claude/projects/<project>/memory/. MEMORY.md is an index; the first 200 lines or 25 KB load at session start

Machine-local

All worktrees of a repo share one folder. Nothing is shared with teammates. On by default locally; off in self-hosted environments

Ask for it

"remember we use the SQLite file backend/todo.db" goes to memory. "add this to CLAUDE.md" goes to the file

/memory

Lists memory files, opens them, has the on/off toggle. "autoMemoryEnabled": false switches it off for a project

Auto memory on disk: a simple example

WHERE IT IS SAVED

```
macOS / Linux
~/.claude/projects/<project>/memory/

Windows
C:\Users\<you>\.claude\projects\<project>\memory
\
```

FOLDER CONTENTS

```
memory/
├── MEMORY.md    # index, loads every session
├── feedback_short_answers.md
└── project_sqlite_db.md
```

MEMORY.MD (THE INDEX)

```
- [Short answers](feedback_short_answers.md):
  no summary at the end
- [SQLite database](project_sqlite_db.md):
  backend/todo.db, created on first run
```

FEEDBACK_SHORT_ANSWERS.MD (ONE NOTE)

```
---
type: feedback
modified: 2026-11-06T10:15:00Z
---
Prefers short answers. Don't add a summary
at the end of a response.
```

Illustrative. Only MEMORY.md loads at start; Claude opens a note file when it needs it.

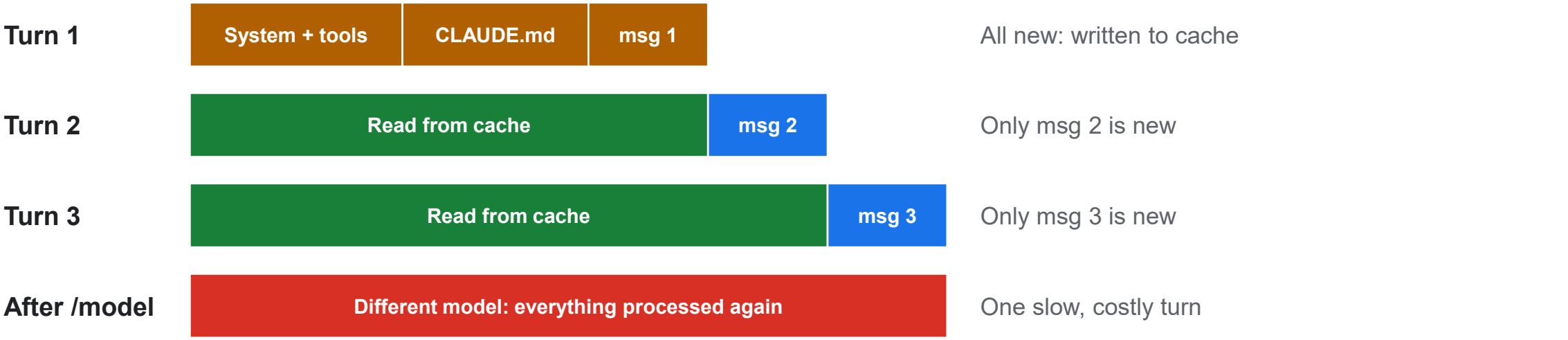
The context budget: what loads, and when

WHAT LOADS	WHEN	COST CONTROL
Instruction files, AGENTS.md, unscoped rules	Every session start	Keep them short; scope rules with paths:
Path rules, subfolder CLAUDE.md / AGENTS.md	When matching files are touched	Already on demand
Skill descriptions	Session start (full text only when used)	Skills over big instruction files (M6)
MCP tool definitions	Names only, until used	Disable unused servers (M10)
Subagent work	Own context window; only a summary returns	Delegate bulky work (M7)

M01 recap: /context shows the bar, /compact frees space with a focus, /clear starts fresh. Here we control the always-loaded part.

Prompt caching: cheaper repeated turns

Every turn re-sends everything. Caching means the unchanged start is read back cheaply instead of processed again.



about 1/10

price of a cached input token vs a normal one

1 hour

cache lifetime on a Claude plan (main conversation)

5 minutes

with an API key, cloud provider or usage credits

What breaks the cache, and what keeps it

Breaks it (next turn is slow and costly)

- Switching model with /model
- Changing effort (on older models)
- Turning on fast mode
- /compact (rebuilds a shorter conversation)
- A break longer than the cache lifetime
- Upgrading Claude Code (first session after)

Keeps it

- Editing files in the repo
- Editing CLAUDE.md mid-session
- Using skills and commands
- Changing permission mode
- /rewind to an earlier turn
- Starting a subagent

An edited CLAUDE.md keeps the cache because it is NOT reloaded: it applies after /clear, /compact or a restart.

Check it: `/usage` shows a Prompt cache line: share of input from cache, misses, and warm or cold.

Your turn: where does each go?

Choose instruction file, path rule, skill, hook / permission, or auto memory for each.

1. "Run the linter before every commit, no exceptions."
2. "I prefer short answers."
3. "The 12-step procedure to publish a release."
4. "Files under migrations/ follow the expand-then-contract pattern."
5. "The app is Python 3.12; tests run with uv run pytest -q."
6. "The frontend/ team's React conventions, shared with other coding tools."

REFERENCE

Example: project instruction file (to-do app)

```
# Project: todo-app

## Commands
- Backend: `cd backend`, `uv sync`, `uv run uvicorn app.main:app --reload`
- Test: `uv run pytest -q` in backend/ (all tests must pass before a change is done)
- Frontend: `cd frontend`, `npm install`, `npm run dev`

## Layout
- `backend/app/main.py` routes only. Business rules live in `backend/app/rules.py`.
- `backend/app/db.py` is the only place that talks to SQLite (`backend/todo.db`).

## Rules
- Dates are UTC, ISO 8601. A title is 1-200 characters after trimming.
- Completed to-dos are archived, never deleted. Never log titles; log the id.
- Every new endpoint needs a test in `backend/tests/`.

## Never
- Do not edit `uv.lock` by hand. Do not add a dependency without asking.
```

REFERENCE

Example: path rule and layered files

PATH RULE (FRONTEND)

```
---
paths:
  - "frontend/src/**/*.{js,jsx}"
---

- Components are function
  components with hooks.
- Only frontend/src/api.js
  calls fetch.
```

LAYERED FILES

```
~/.claude/CLAUDE.md
   personal: "explain before editing"
todo-app/AGENTS.md
   team rules shared with other tools
todo-app/CLAUDE.md
   "@AGENTS.md" plus Claude-only lines
todo-app/.claude/rules/tests.md
   paths: ["backend/tests/**"]
todo-app/backend/AGENTS.md
   loads when Claude touches backend/
todo-app/frontend/CLAUDE.md
   if present, used instead of frontend/AGENTS.md
```

REFERENCE

Commands

COMMAND	DOES
`/init`	Drafts a starter CLAUDE.md from the repo, or suggests improvements to an existing one. Reads Cursor and Copilot rules too
`/memory`	List, open and edit memory files; toggle auto memory
`/context`	See what loaded, under Memory files
`/config`	Set Project instructions (which of CLAUDE.md / AGENTS.md load)
`/doctor prompt-audit`	Audit instruction files, rules, skills and subagents

/init is a starting point. Delete anything Claude could learn by reading the code; keep pitfalls, reasons and conventions that differ from defaults. CLAUDE_CODE_NEW_INIT=1 gives an interactive version that can also set up skills and hooks.

CHECK YOUR UNDERSTANDING

Question 1

Your repo has AGENTS.md and a CLAUDE.md that does not mention it. Which does Claude read?

Think first, then we discuss.

CHECK YOUR UNDERSTANDING

Question 2

backend/ has its own AGENTS.md. When does Claude read it, and when does it never read it?

Think first, then we discuss.

CHECK YOUR UNDERSTANDING

Question 3

You split a 400-line file into four @ imports of 100 lines each. Did the context cost go down?

Think first, then we discuss.

CHECK YOUR UNDERSTANDING

Question 4

You typed "always answer in British English" in chat three hours ago and it stopped working after a /compact. Why, and what is the fix?

Think first, then we discuss.

CHECK YOUR UNDERSTANDING

Question 5

What decides whether a rule with paths: is loaded?

Think first, then we discuss.

CHECK YOUR UNDERSTANDING

Question 6

A teammate wants Claude to remember the team's branch naming convention. Auto memory or instruction file?

Think first, then we discuss.